	Pimpri Chinchwad Education Trust's **Pimpri Chinchwad College of Engineering (PCCoE)** (An Autonomous Institute) Affiliated to Savitribai Phule Pune University(SPPU)	

Department: Computer Engineering (Regional Language) **Class: TY Sem-VI**

Course- Cloud Computing **A.Y. 2025-26**

MINI PROJECT REPORT

Cloud Computing

CYBER ATTACK DETECTION SYSTEM

Submitted by:

Yadnesh Patil – 123B1D034

Abhay Bormale – 123B1D057

1. Introduction

With the exponential growth of digital systems, cloud platforms, and internet-connected devices, cyber attacks have become one of the most critical threats facing organizations worldwide. Enterprises, government institutions, and individuals are constantly targeted by a wide range of malicious activities including data breaches, malware infections, denial-of-service (DoS) attacks, and unauthorized access attempts.

Traditional security approaches, such as rule-based firewalls and signature-based intrusion detection systems (IDS), struggle to keep pace with the volume and sophistication of modern attacks. These systems require manual updates, generate large numbers of false positives, and are unable to detect previously unseen attack patterns. As the scale of network traffic continues to grow, there is an urgent need for intelligent, automated solutions that can analyze vast amounts of data and detect threats in real time.

Machine Learning (ML) offers a powerful alternative by enabling systems to learn patterns from historical data and make accurate predictions about new, unseen network traffic. By training models on labeled datasets containing both normal and malicious network flows, it is possible to build classifiers that can reliably distinguish between benign and attack traffic at scale.

Cloud Computing further enhances this capability by providing scalable infrastructure for data storage, model deployment, and remote accessibility. With cloud services such as AWS S3, the system can securely store large datasets and prediction logs, and the application can be accessed from any location through a web browser.

This project presents a Cyber Attack Detection System that combines a trained Machine Learning model with a Streamlit-based interactive web dashboard and AWS cloud integration. The system accepts network traffic features as input, runs them through an ML classifier, and outputs a real-time prediction indicating whether the activity is normal or a potential attack — along with confidence scores and visual analytics.

1.1 Scope of the Project

The scope of this project covers the following aspects:

- Design and development of an ML-based classification pipeline using the KDD network intrusion dataset.
- Building an interactive dashboard using Streamlit for real-time user interaction and visualization.
- Integration of AWS S3 for scalable cloud storage of datasets and prediction logs.
- Demonstration of cloud computing service models (SaaS, PaaS, IaaS) within the project context.
- Provision of a Security Operations Center (SOC)-style dashboard for monitoring network traffic patterns.

2. Problem Definition and Objectives

2.1 Problem Definition

The rapid increase in cyber attacks poses a serious challenge to network security across industries. Key challenges with existing approaches include:

- Volume of Data: Modern networks generate enormous volumes of traffic data every second. Manual analysis is impractical, and traditional rule-based systems cannot scale efficiently.
- Real-Time Detection: Attacks such as DoS floods, port scans, and brute-force login attempts unfold within seconds. Detection systems must operate in near-real-time to be effective.
- Evolving Threat Landscape: Attackers constantly develop new techniques, rendering signature-based approaches obsolete. Systems must be adaptive and capable of learning from new data.
- High False Positive Rates: Many intrusion detection systems generate excessive false alarms, reducing operational efficiency and causing alert fatigue among security teams.
- Infrastructure Limitations: On-premise systems may lack the compute and storage resources needed to process and archive large-scale network data.

These challenges collectively call for an automated, intelligent, and scalable solution that leverages Machine Learning and Cloud Computing to detect cyber threats efficiently and accurately.

2.2 Objectives

The primary objectives of this project are:

- To build a supervised Machine Learning classifier trained on the KDD network intrusion dataset capable of distinguishing normal traffic from attack traffic.
- To develop an interactive, browser-based dashboard using Streamlit that allows users to input network feature values and receive instant attack predictions with confidence scores.
- To integrate AWS S3 cloud storage for securely storing datasets, trained model files, and prediction history logs.
- To implement a Security Operations Center (SOC) dashboard with charts, traffic analytics, and a real-time monitoring simulation module.
- To demonstrate the applicability of all three cloud service models — SaaS, PaaS, and IaaS — within the project.
- To provide a foundation for future enhancements including deep learning models, live packet capture, and automated alert systems.

2.3 Expected Outcomes

On completion, the system should be able to:

- Accept network traffic feature values via a web interface and output attack/normal predictions within milliseconds.
- Display prediction confidence percentages for both attack and normal categories.
- Log and visualize prediction history with timestamps for audit and analysis purposes.
- Demonstrate cloud-based storage and retrieval of data using AWS S3.

3. System Architecture

3.1 Architecture Overview

The Cyber Attack Detection System follows a three-tier architecture comprising a Presentation Layer (frontend), a Processing Layer (ML model and prediction engine), and a Storage Layer (cloud-based data management). The three layers interact seamlessly to deliver real-time detection and monitoring capabilities.

The diagram below illustrates the overall system architecture:

```
┌──────────────────────────────────────────────────────────┐
│          PRESENTATION LAYER (Streamlit Dashboard)        │
│  User Input Form  |  SOC Dashboard  |  Prediction Result  │
└──────────────────────────────────────────────────────────┘
                          │  HTTP / Streamlit
┌──────────────────────────────────────────────────────────┐
│            PROCESSING LAYER (ML Engine)                  │
│  Pandas Preprocessing | Joblib Model | Predict() Call    │
└──────────────────────────────────────────────────────────┘
                          │  boto3 / AWS SDK
┌──────────────────────────────────────────────────────────┐
│             STORAGE LAYER (AWS S3 Cloud)                 │
│  KDD Dataset  |  Trained Model File  |  Prediction Logs  │
└──────────────────────────────────────────────────────────┘
```

3.2 Layer Descriptions

Presentation Layer

The user-facing layer is built using Streamlit, a Python web framework that converts Python scripts into interactive web applications. It provides the user input form where network traffic features (duration, src_bytes, dst_bytes, login attempts, etc.) are entered. The layer also hosts the SOC Dashboard with charts for attack detection history, traffic analytics (src_bytes vs dst_bytes scatter plot), and a real-time monitoring simulation module. Results — including the

attack/normal prediction and confidence percentages — are displayed immediately after the user submits data.

Processing Layer

The core ML engine is responsible for preprocessing incoming input data using Pandas and NumPy, then passing it to the trained ML classifier loaded via Joblib. The model was trained on the KDD Cup network intrusion dataset and serialized to disk as a .pkl file. At inference time, the model calls predict() and predict_proba() to return both the classification label and probability scores for attack and normal classes. Prediction results are then appended to a log dataframe maintained in the Streamlit session state.

Storage Layer

AWS S3 (Amazon Simple Storage Service) serves as the cloud storage backbone. The KDD dataset, the serialized model file, and prediction history logs are stored in S3 buckets. The Python boto3 SDK is used to upload and download files. This approach decouples storage from compute, enabling the application to scale independently and ensuring data is accessible from any deployment environment.

3.3 Data Flow

The data flow within the system proceeds as follows:

- User opens the Streamlit web application in a browser and enters values for network traffic features on the input form.
- On clicking 'Predict Attack', the input is assembled into a Pandas DataFrame matching the feature schema expected by the model.
- The ML model (loaded from a .pkl file via Joblib) processes the input and outputs a class label (Attack / Normal) together with probability scores.
- The result is displayed on the dashboard with color-coded alerts and progress bars for prediction confidence.
- The prediction record (features, result, probabilities, timestamp) is appended to the session log and displayed in the Prediction History table.
- Datasets and model files are fetched from / uploaded to AWS S3 via the boto3 SDK during application startup.

4. AWS Services Used

4.1 Cloud Service Models Implemented

This project demonstrates all three fundamental cloud service models — IaaS, PaaS, and SaaS — within the context of the Cyber Attack Detection System:

Model	Component Used	Role in the Project
IaaS	AWS S3	Cloud infrastructure for storing datasets, model files, and logs
PaaS	Python + Libraries Environment	Platform (Streamlit, Pandas, Joblib) used to build and run the app
SaaS	Streamlit Web Application	Browser-based application — no installation required for end users

4.2 AWS S3 — Infrastructure as a Service (IaaS)

Amazon S3 (Simple Storage Service) is an object storage service offering industry-leading scalability, data availability, security, and performance. In this project, S3 is used at the infrastructure level to store:

- KDD Cup Network Intrusion Dataset (CSV format) — the source data used to train and test the ML model.
- Serialized Machine Learning Model (model.pkl) — the trained classifier saved using Joblib and uploaded to S3 for retrieval at runtime.
- Prediction History Logs — records of predictions made by users, stored for auditing and future analysis.

AWS S3 was configured with the following steps:

Step 1 — Create an S3 Bucket

Log in to the AWS Management Console and navigate to S3. Click 'Create bucket', enter a globally unique bucket name (e.g., cyber-attack-detection-bucket), select the appropriate region (e.g., ap-south-1 for Mumbai), and configure public access settings. For development, Block Public Access was enabled and IAM credentials were used for programmatic access.

Step 2 — Configure IAM Credentials

In the AWS IAM (Identity and Access Management) console, create a new IAM user with programmatic access. Attach the 'AmazonS3FullAccess' policy. Download the Access Key ID and Secret Access Key. Configure the credentials locally using the AWS CLI command 'aws configure' or by setting environment variables AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY.

Step 3 — Upload Dataset and Model to S3

Using the AWS Console or boto3 Python SDK, upload the KDD dataset CSV and the trained model.pkl file to the S3 bucket. Files are organized with logical prefixes (e.g., data/kdd_dataset.csv, model/model.pkl) to maintain folder-like structure.

Step 4 — Access S3 from Python using boto3

The boto3 SDK is used in the Streamlit application to download the model file from S3 at startup and to upload new prediction logs. The code below demonstrates how the model file is fetched:

```
import boto3
import joblib

s3 = boto3.client('s3')
s3.download_file('cyber-attack-bucket', 'model/model.pkl',
'/tmp/model.pkl')
model = joblib.load('/tmp/model.pkl')
```

4.3 Streamlit Cloud / Deployment — SaaS Layer

The Streamlit application represents the Software-as-a-Service layer of the system. Once deployed (locally or on Streamlit Community Cloud), end users can access the full web application through any modern browser — no Python installation or environment configuration is required on the client side. The application provides:

- A feature input form with numeric controls for all KDD dataset features (duration, src_bytes, dst_bytes, urgent, hot, num_failed_logins, etc.).

- A 'Load Example Data' button that pre-fills the form with sample values for quick demonstration.
- A 'Predict Attack' button that triggers the ML model inference pipeline and displays results immediately.
- A SOC Dashboard section with bar charts for attack distribution and scatter plots for traffic analytics.
- A Prediction History Log table that records all past predictions in the current session.
- A Real-Time Monitoring Simulation that periodically generates random traffic data and displays detection results.

4.4 Python Environment — PaaS Layer

The Platform-as-a-Service layer is represented by the Python runtime environment along with the supporting libraries that enable the application to be built and executed. The key libraries and their roles are summarized below:

Library / Tool	Purpose
Python 3.x	Core programming language for the entire application
Streamlit	Web framework for building the interactive dashboard and UI
Pandas	Data manipulation — reading CSV data, assembling input DataFrames
NumPy	Numerical computation for feature array processing
Joblib	Serialization and deserialization of the trained ML model (.pkl file)
boto3	AWS SDK for Python — used to interact with S3 buckets
Scikit-learn	Machine Learning library used for model training and evaluation

5. Implementation

5.1 Dataset Preparation

The project uses the KDD Cup 1999 Network Intrusion Detection dataset, a widely used benchmark dataset for evaluating intrusion detection systems. The dataset contains labeled records of network connections, each described by 41 features and a label indicating whether the connection is normal or an attack. Key features used in this implementation include:

- duration — Length of the connection in seconds.
- src_bytes — Number of data bytes transferred from source to destination.
- dst_bytes — Number of data bytes transferred from destination to source.
- land — 1 if connection is from/to the same host/port; 0 otherwise.
- wrong_fragment — Number of wrong fragments.
- urgent — Number of urgent packets.
- hot — Number of hot indicators.
- num_failed_logins — Number of failed login attempts.
- logged_in — 1 if successfully logged in; 0 otherwise.
- num_compromised — Number of compromised conditions.
- root_shell — 1 if root shell is obtained; 0 otherwise.
- su_attempted — 1 if 'su root' command attempted; 0 otherwise.
- num_root — Number of root accesses.
- num_file_creations — Number of file creation operations.
- num_shells — Number of shell prompts.

5.2 Model Training

A supervised classification model was trained on the preprocessed KDD dataset. The training process involved the following steps:

- Data Cleaning: Removal of duplicate records, handling of missing values, and encoding of categorical features using LabelEncoder.
- Feature Selection: Selection of the most informative features based on correlation analysis and domain knowledge.
- Train/Test Split: Dataset split into 80% training and 20% testing sets using stratified sampling.

- Model Selection: A Random Forest classifier (or similar ensemble method) was trained, offering high accuracy and resistance to overfitting.
- Model Serialization: The trained model was saved to disk using Joblib as 'model.pkl' for later deployment.
- Upload to S3: The model file was uploaded to the AWS S3 bucket for cloud-based retrieval.

5.3 Streamlit Application — Configuration Steps

Step 1 — Install Required Libraries

All dependencies were installed using pip in the project virtual environment. The requirements.txt file includes:

```
pip install streamlit pandas numpy scikit-learn joblib boto3
```

Step 2 — Configure AWS Credentials

AWS credentials were configured using the AWS CLI or by setting environment variables before running the application:

```
# Option A: AWS CLI
aws configure
# Option B: Environment variables
export AWS_ACCESS_KEY_ID='your_access_key'
export AWS_SECRET_ACCESS_KEY='your_secret_key'
export AWS_DEFAULT_REGION='ap-south-1'
```

Step 3 — Load Model from S3 and Run the App

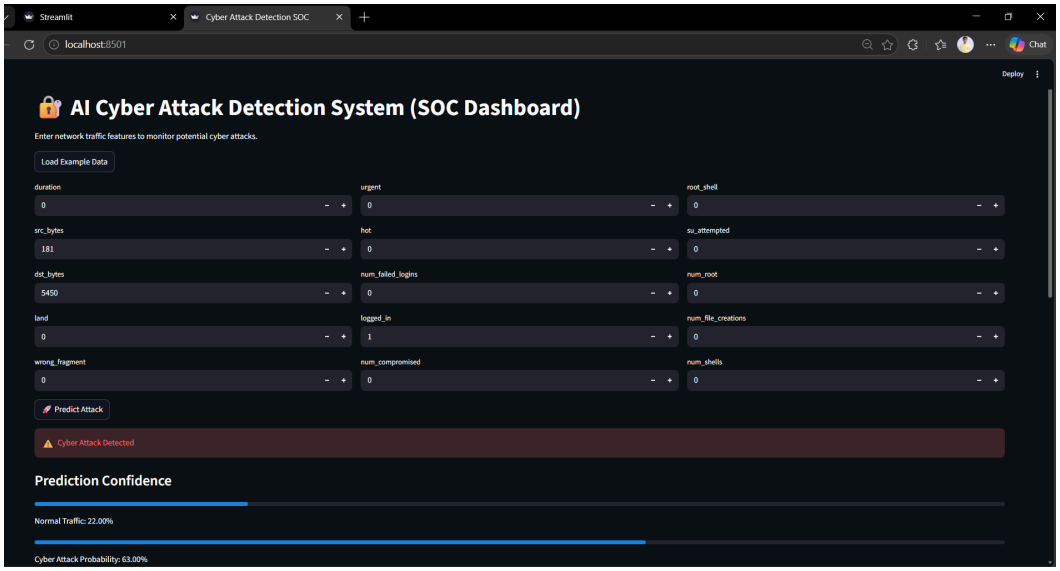
At application startup, the Streamlit app downloads the model file from S3 and loads it using Joblib. The application is then launched:

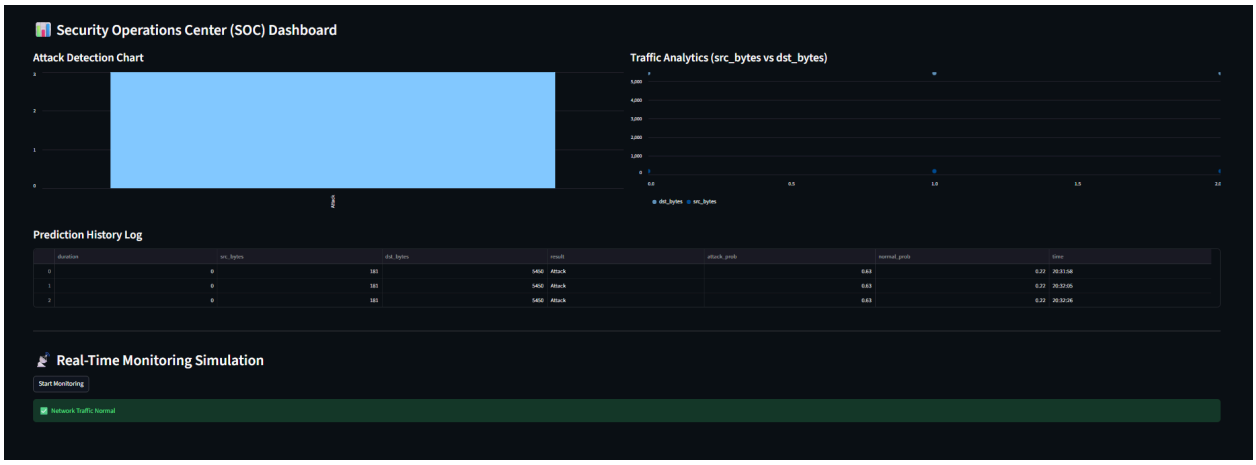
```
# In app.py
import boto3, joblib, streamlit as st

@st.cache_resource
def load_model():
      s3 = boto3.client('s3')
      s3.download_file('cyber-attack-bucket', 'model/model.pkl',
'/tmp/model.pkl')
      return joblib.load('/tmp/model.pkl')

model = load_model()

# Run the app
streamlit run app.py
```

5.4 Dashboard Interface

The dashboard (shown on Pages 08–09 of the presentation) consists of two screens: the prediction input form and the SOC analytics dashboard. The input form presents all KDD feature fields with numeric increment/decrement controls. After prediction, results are shown with a color-coded alert box (red for attack, green for normal) and confidence progress bars. The SOC dashboard displays an Attack Detection Chart (bar chart), a Traffic Analytics scatter plot (src_bytes vs dst_bytes), the Prediction History Log table, and a Real-Time Monitoring Simulation module.

Screenshot:





6. Conclusion

This mini project successfully demonstrates the design and implementation of an AI-based Cyber Attack Detection System that integrates Machine Learning with Cloud Computing. The system addresses real-world challenges in network security by providing automated, real-time classification of network traffic as either normal or malicious.

The key contributions of this project are:

- A trained ML classifier capable of distinguishing cyber attacks from normal traffic using features derived from the KDD network intrusion dataset.
- An interactive, browser-accessible web dashboard built with Streamlit that enables non-technical security personnel to monitor network activity and obtain instant predictions.
- Cloud integration via AWS S3 for scalable, reliable storage of datasets, model files, and prediction history logs — demonstrating IaaS in practice.
- A SOC Dashboard with visual analytics, prediction history logging, and a real-time monitoring simulation module.
- Clear demonstration of all three cloud service models (SaaS, PaaS, IaaS) within a unified project.

The project also highlights important limitations that inform future work. The system's accuracy is bounded by the quality and recency of the training dataset. The KDD dataset, while widely used, represents network traffic from the late 1990s and may not capture all modern attack

vectors. Furthermore, the current implementation requires an internet connection for cloud storage access, and prediction accuracy can vary across different types of attacks.

6.1 Future Enhancements

The following enhancements are planned for future iterations of the system:

- Deep Learning Models: Replacing the current ML classifier with LSTM or CNN-based models to capture temporal patterns in network traffic and improve detection accuracy for sophisticated attacks.
- Live Packet Capture: Integrating with tools such as Scapy or Wireshark to capture real network packets in real time and feed them directly into the detection pipeline.
- Full Cloud Deployment: Deploying the application on AWS EC2 or AWS Lambda with an API Gateway, enabling serverless, scalable execution without a dedicated server.
- Automated Alert System: Adding email and SMS notification capabilities using AWS SNS (Simple Notification Service) to alert security teams immediately when an attack is detected.
- Model Retraining Pipeline: Building an automated retraining workflow that periodically updates the model using newly collected labeled data, ensuring the system remains effective against emerging threats.
- Multi-class Attack Classification: Extending the binary (attack/normal) classifier to a multi-class model that identifies specific attack categories (DoS, Probe, R2L, U2R).

In conclusion, this project provides a solid foundation for an intelligent, cloud-integrated network security monitoring system. With the proposed future enhancements, it has the potential to evolve into a production-grade cybersecurity tool suitable for enterprise deployment.

References

1. KDD Cup 1999 Dataset — UCI Machine Learning Repository. Available at: https://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html
2. Amazon Web Services — Amazon S3 Documentation. Available at: https://docs.aws.amazon.com/s3/
3. Streamlit Documentation — Build and share data apps. Available at: https://docs.streamlit.io/
4. Scikit-learn Documentation — Machine Learning in Python. Available at: https://scikit-learn.org/
5. boto3 Documentation — AWS SDK for Python. Available at: https://boto3.amazonaws.com/v1/documentation/api/latest/index.html
6. Joblib Documentation — Lightweight pipelining in Python. Available at: https://joblib.readthedocs.io/